Application of Machine Learning Methods to Predict the Level of Building Damage in the Nepal Earthquake

UE24CS352A Machine Learning mini-project | Team 18, Problem 58 | Syed Ayaan Hasan (PES2UG24CS545), Sriniket Deeduvanu (PES2UG24CS521)

1. Problem statement

Assessing earthquake damage building by building is slow. Given a building's structure, age, use and location, we predict the damage grade it suffered in the April 2015 Gorkha earthquake in Nepal: 1 (low damage), 2 (medium damage) or 3 (almost complete destruction). This is a three-class classification problem scored with micro-averaged F1, following the DrivenData competition "Richter's Predictor: Modeling Earthquake Damage" on which the problem is based.

2. Dataset

The data comes from the 2015 Nepal Earthquake Open Data Portal through the DrivenData competition. The training file has 260,601 buildings and 38 features: 8 categorical (letter-coded) and 30 numeric, including floor count, age, area and height percentages, 11 superstructure-material flags, secondary-use flags and three hierarchical location IDs (*geo_level_1/2/3_id*, with 31, 1,414 and 11,595 values). There are no missing values. 6.2% of rows repeat the features of another building and were kept, and 1,496 buildings have a recorded age of 200 years or more (maximum 995), which looks like a placeholder. The classes are imbalanced (9.6% grade 1, 56.9% grade 2, 33.5% grade 3), so always predicting grade 2 gives a micro-F1 of 0.5689. The competition test file (86,868 buildings) has no public labels, so we split the training file once into 80% training and 20% validation (stratified, seed 42: 208,480 and 52,121 buildings) and report validation scores for every model.

3. Approach

We compared four model families on the same split: K-nearest neighbours (KNN), a feed-forward neural network, a random forest and gradient boosting. Two preprocessing pipelines were used. For KNN and the neural network, categorical features are one-hot encoded, numeric features standardised, *age* capped at 100, and *geo_level_2_id* and *geo_level_3_id* dropped (one-hot encoding them would add about 13,000 columns). The tree models use ordinal-coded categoricals, the raw age and all three geo IDs as numbers. Preprocessing is fitted on the training part only. Hyperparameters were chosen by validation micro-F1: *k* (1 to 51) and vote weighting for KNN; architecture (10 units up to 256-128-64) and L2 strength for the network, with three random seeds for the final model; number of trees, depth, leaf size and class weights for the forest (8 configurations); learning rate, iterations and leaves per tree for boosting (9 configurations, early stopping on an internal 10% of the training data).

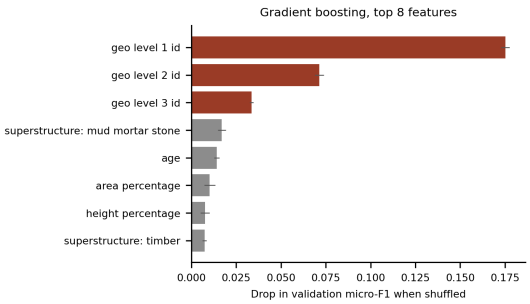
4. Implementation overview

Everything is written in Python with scikit-learn. *src/data.py* holds the shared loading, split and preprocessors and *src/evaluate.py* the metrics, confusion matrices and permutation importance. Six notebooks cover the EDA, KNN, neural network, random forest, gradient boosting and the final comparison. Each model notebook saves its metrics as JSON in *results/metrics/*, and the comparison notebook only reads those files, so all four models are compared from the same saved numbers. *main.py* retrains the final model and prints its validation report. The code is in a private GitHub repository with a README describing the setup.

5. Results

Model	Selected setting	micro-F1	macro-F1	Recall, grades 1 / 2 / 3
Gradient boosting	learning rate 0.1, 63 leaves, stopped at 350 iterations	**0.7459**	**0.6929**	0.52 / 0.85 / 0.63
Random forest	300 trees, minimum leaf size 2	0.7314	0.6645	0.45 / 0.86 / 0.59
Neural network	hidden layers 256-128-64, L2 0.01	0.6972	0.6328	0.44 / 0.81 / 0.57
KNN	k = 51, uniform weights	0.6802	0.6066	0.40 / 0.82 / 0.52
Baseline	always predict grade 2	0.5689	-	-

Gradient boosting is best on both micro- and macro-F1. Its lead over the random forest (0.0145) is several times the roughly 0.002 sampling error of the validation set. More capacity helped both tree models (63 leaves instead of 31 raised boosting from 0.7343 to 0.7413 at 250 iterations; forest leaf size 2 instead of 5 raised 0.7239 to 0.7307), but leaf size 1 overfitted (training 0.987, validation 0.7234) and class weighting lowered micro-F1 (0.7239 to 0.6743).



Permutation importance shows that location dominates: *geo_level_1_id* is by far the most important feature for both tree models (a micro-F1 drop of 0.175 for boosting and 0.142 for the forest when shuffled), followed by *geo_level_2_id*, *geo_level_3_id* and the mud-mortar-stone material flag.

6. Conclusions

Gradient boosting is our final model, with a validation micro-F1 of 0.7459, 0.177 above the always-grade-2 baseline. The tree models beat KNN and the neural network by 3 to 7 points, most likely because they can use the fine-grained location IDs that the other two had to drop. Low damage remains the hard class: even boosting recalls only 52% of grade-1 buildings, and most errors are predictions of grade 2, while confusing grades 1 and 3 is rare (at most 118 buildings for any model). The main limitations are that hyperparameters were chosen on the same validation split that is reported, that there is a single split, that duplicate buildings can favour high-capacity models, and that we did not score the competition test set. Possible next steps are encoding *geo_level_2_id* and *geo_level_3_id* for the neural network (target encoding or embeddings), tuning the decision rule to raise grade-1 recall, and cross-validation.